studocu

R - practical - Lab record

M.Sc Computer Science (Bharathiar University)

Scan to open on Studocu

Studocu is not sponsored or endorsed by any college or university
Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

L.R.G. GOVERNMENT ARTS COLLEGE FOR WOMEN

(Affiliated To Bharathiar University)

TIRUPUR – 641 604.

DEPARTMENT OF COMPUTER SCIENCE

I MSC COMPUTER SCIENCE

PRACTICAL III: DATA MINING LAB USING R

NAME :

REG. NO :

CLASS : I M.SC. COMPUTER SCIENCE

SUBJECT : DATA MINING LAB USING R

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

L.R.G. GOVERNMENT ARTS COLLEGE FOR WOMEN

(Affiliated To Bharathiar University)

TIRUPUR – 641 604.

Name :

Register No :

Class : I M.Sc. Computer Science

This is to certify that it is a Bonafide record of work done by the above Student of the M.Sc. Computer Science. **PRACTICAL III: DATA MINING LAB USING R** during the academic year 2024-2025.

Staff in Charge **Head of the Department**

Submitted for the Bharathiar University Practical Exam held on …….

Internal Examiner **External Examiner**

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

INDEX

S. NO.	DATE	CONTENT	PAGE NO.	SIGNATURE
1		APRIORI ALGORITHM TO EXTRACT ASSOCIATION RULES OF DATA MINING		
2		K-MEANS CLUSTERING ALGORITHM		
3		HIERARCHICAL CLUSTERING		
4		CLASSIFICATION ALGORITHM		
5		DECISION TREE		
6		LINEAR REGRESSION		
7		DATA VISUALIZATION		

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

EX. NO: 01	APRIORI ALGORITHM TO EXTRACT ASSOCIATION RULES OF DATA MINING	PAGE NO:
DATE :		

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

1. APRIORI ALGORITHM TO EXTRACT ASSOCIATION RULES OF DATA MINING

```
#Apriori Algorithm
#Load the arules
library(arules)
#Load library dataset
library(arulesViz)
library(datasets)
#Load the data Groceries
data=Groceries
#Using apriori() function to print the several data set
head(data)
trules=apriori(data,parameter=list(supp=0.001,conf=0.3,minlen=2))
#Using inspect() function to print the associated items
inspect(trules[1:10])
brules=apriori(data,parameter=list(supp=0.001,conf=0.8),appearance=list(default="lhs",
rhs=c("whole milk")))
inspect(head(brules))
acrules=apriori(data,parameter=list(supp=0.001,conf=0.8),appearance=list(default="rhs",
lhs=c("curd","cereals")))
inspect(acrules)
inspect(sort(brules[1:10]))
plot(trules)
plot(brules[1:50],method="graph")
plot(acrules,method="graph")
```

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

OUTPUT:

```
library(arules)
> head(data)
transactions in sparse format with
6 transactions (rows) and
169 items (columns)
> trules=apriori(data,parameter=list(supp=0.001,conf=0.3,minlen=2))
Apriori
Parameter specification:
confidence minval smax arem aval originalSupport maxtime support minlen
    0.3    0.1    1 none FALSE            TRUE       5   0.001      2
maxlen target   ext
    10 rules FALSE
Algorithmic control:
filter tree heap memopt load sort verbose
    0.1 TRUE TRUE FALSE TRUE    2    TRUE
Absolute minimum support count: 9
set item appearances ... [0 item(s)] done [0.00s].
set transactions ... [169 item(s), 9835 transaction(s)] done [0.00s].
sorting and recoding items ... [157 item(s)] done [0.00s].
creating transaction tree ... done [0.00s].
checking subsets of size 1 2 3 4 5 6 done [0.01s].
writing ... [13770 rule(s)] done [0.01s].
creating S4 object ... done [0.00s].
```

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> inspect(trules[1:10])

```
    lhs                 rhs          support     confidence   lift     count
[1]  {honey}                    => {whole milk} 0.001118454 0.7333333  2.870009 11
[2]  {soap}                     => {whole milk} 0.001118454 0.4230769  1.655775 11
[3]  {tidbits}                  => {soda}       0.001016777 0.4347826  2.493345 10
[4]  {tidbits}                  => {rolls/buns} 0.001220132 0.5217391  2.836542 12
[5]  {cocoa drinks}             => {whole milk} 0.001321810 0.5909091  2.312611 13
[6]  {snack products}           => {soda}       0.001118454 0.3666667  2.102721 11
[7]  {snack products}           => {rolls/buns} 0.001118454 0.3666667  1.993459 11
[8]  {pudding powder}           => {whole milk} 0.001321810 0.5652174  2.212062 13
[9]  {cooking chocolate}        => {whole milk} 0.001321810 0.5200000  2.035097 13
[10] {bathroom cleaner}         => {soda}       0.001016777 0.3703704  2.123961 10
```

brules=apriori(data,parameter=list(supp=0.001,conf=0.8),appearance=list(default="lhs", rhs=c("whole milk")))

Apriori

Parameter specification:

```
confidence minval smax arem aval originalSupport maxtime support minlen
    0.8    0.1    1 none FALSE            TRUE       5   0.001      1
maxlen target   ext
    10 rules FALSE
```

Algorithmic control:

```
filter tree heap memopt load sort verbose
    0.1 TRUE TRUE FALSE TRUE    2    TRUE
```

Absolute minimum support count: 9

set item appearances ... [1 item(s)] done [0.00s].

set transactions ... [169 item(s), 9835 transaction(s)] done [0.00s].

sorting and recoding items ... [157 item(s)] done [0.00s].

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

```
creating transaction tree ... done [0.00s].
checking subsets of size 1 2 3 4 5 6 done [0.01s].
writing ... [252 rule(s)] done [0.00s].
creating S4 object ... done [0.00s].
> inspect(head(brules))
    lhs                    rhs            support     confidence     lift     count
[1] {curd, cereals}        => {whole milk} 0.001016777  0.9090909 3.557863    10
[2] {yogurt, cereals}      => {whole milk} 0.001728521  0.8095238 3.168192    17
[3] {butter, jam}          => {whole milk} 0.001016777  0.8333333 3.261374    10
[4] {soups, bottled beer}  => {whole milk} 0.001118454  0.9166667 3.587512    11
[5] {napkins, house keeping products} => {whole milk} 0.001321810 0.8125000 3.179840    13
[6] {whipped/sour cream, house keeping products} => {whole milk} 0.001220132   0.9230769
3.612599    12
>acrules=apriori(data,parameter=list(supp=0.001,conf=0.8),appearance=list(default="rhs"
,lhs=c("curd","cereals")))
Apriori
Parameter specification:
confidence minval smax arem aval originalSupport maxtime support minlen
     0.8    0.1    1 none FALSE            TRUE       5   0.001      1
 maxlen target ext
     10 rules FALSE
Algorithmic control:
filter tree heap memopt load sort verbose
      0.1 TRUE TRUE FALSE TRUE    2    TRUE
Absolute minimum support count: 9
set item appearances ... [2 item(s)] done [0.00s].
set transactions … [169 item(s), 9835 transaction(s)] done [0.00s].
```

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

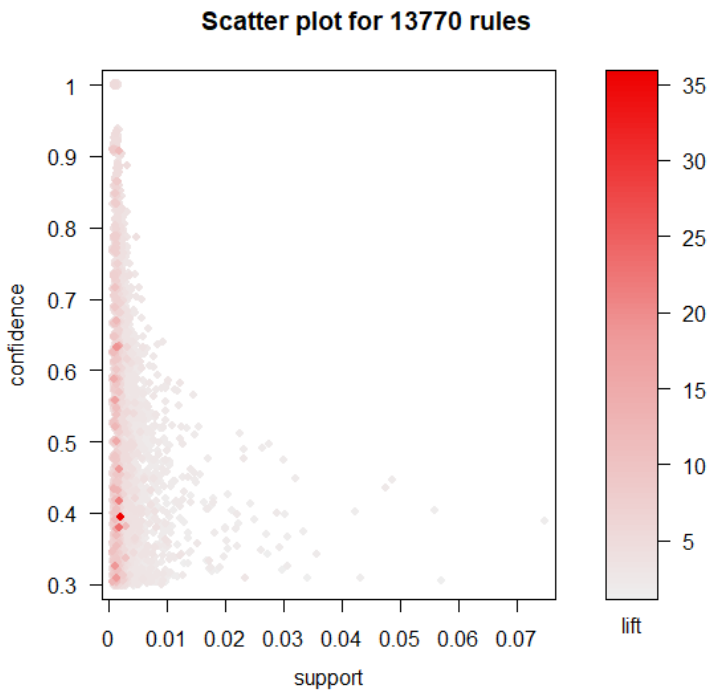
```
sorting and recoding items ... [157 item(s)] done [0.00s].
creating transaction tree ... done [0.00s].
checking subsets of size 1 2 3 done [0.00s].
writing ... [1 rule(s)] done [0.00s].
creating S4 object ... done [0.00s].
> inspect(acrules)
    lhs                 rhs          support     confidence     lift      count
[1] {curd, cereals} => {whole milk} 0.001016777 0.9090909 3.557863 10
> inspect(sort(brules[1:10]))
     lhs                 rhs          support      confidence     lift      count
[1]  {yogurt, cereals}                      => {whole milk} 0.001728521  0.8095238 3.168192    17
[2]  {butter, rice}                         => {whole milk} 0.001525165  0.8333333 3.261374   15
[3]  {napkins, house keeping products} => {whole milk} 0.001321810  0.8125000 3.179840   13
[4]  {whipped/sour cream, house keeping products} => {whole milk} 0.001220132  0.9230769
3.612599    12
[5]  {rice, sugar}                          => {whole milk} 0.001220132  1.0000000 3.913649   12
[6]  {soups, bottled beer}                  => {whole milk} 0.001118454  0.9166667 3.587512   11
[7]  {domestic eggs, rice}                  => {whole milk} 0.001118454  0.8461538 3.311549   11
[8]  {curd, cereals}                        => {whole milk} 0.001016777  0.9090909 3.557863  10
[9]  {butter, jam}                          => {whole milk} 0.001016777  0.8333333 3.261374   10
[10] {pastry, sweet spreads}                => {whole milk} 0.001016777  0.9090909 3.557863   10
```

This document is available on studocu

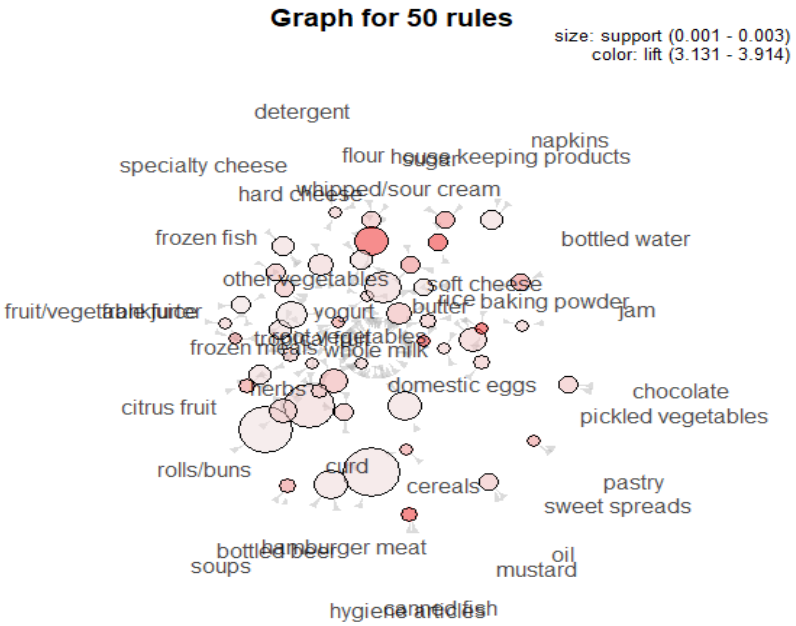
Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> plot(trules)

To reduce overplotting, jitter is added! Use jitter = 0 to prevent jitter.

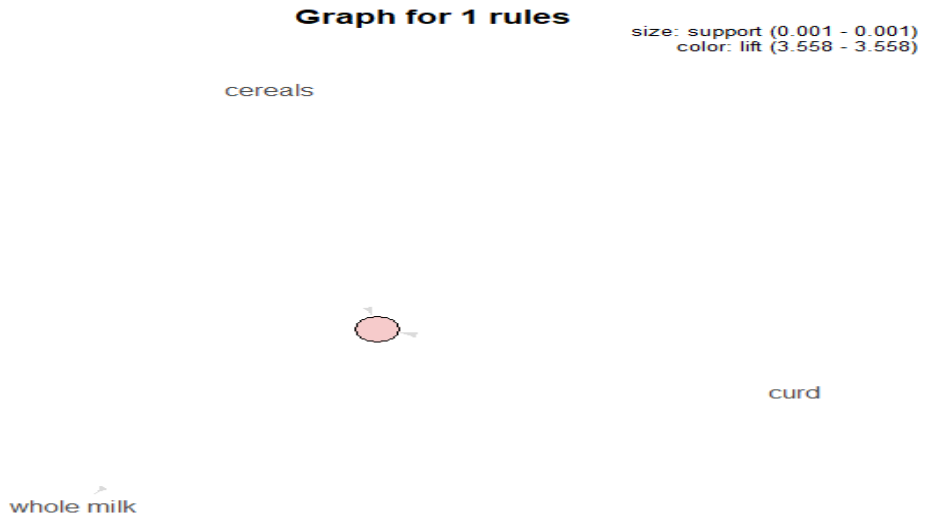


> plot(brules[1:50],method="graph")



Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> plot(acrules,method="graph")



RESULT:

Thus the program was executed successfully and the output is verified.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

EX. NO: 02	K-MEANS CLUSTERING ALGORITHM	PAGE NO:
DATE :		

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

2. K-MEANS CLUSTERING ALGORITHM

```
#K-means Clustering
library(cluster)
df=USArrests
#Number of Rows and Columns of the Actul Data set
dim(df)
head(df)
#remove rows with missing values
df=na.omit(df)
dim(df)
#scale each variable to have mean 0 and sd 1
df=scale(df)
head(df)
set.seed(1)
#Cluster the dataset with 5 groups
km=kmeans(df,centers=5,nstart=25)
print(km)
plot(df)
points(km$centers,col=1:5,pch=8,cex=2)
cnt=table(km$cluster)
print(cnt)
final_data=cbind(df,cluster=km$cluster)
head(final_data)
plot(final_data,cex=0.6,main="Final Data")
ag=aggregate(final_data,by=list(cluster=km$cluster),mean)
head(ag)
plot(ag,cex=0.6,main="Aggregate")
```

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

OUTPUT:

#K-means Clustering

> #Number of Rows and Columns of the Actaul Data set

> dim(df)

[1] 50 4

> head(df)

```
   Murder Assault UrbanPop Rape
Alabama      13.2     236       58 21.2
Alaska       10.0     263       48 44.5
Arizona       8.1     294       80 31.0
Arkansas      8.8     190       50 19.5
California    9.0     276       91 40.6
Colorado      7.9     204       78 38.7
```

> #remove rows with missing values

> dim(df)

[1] 50 4

> #scale each variable to have mean 0 and sd 1

> head(df)

```
     Murder   Assault   UrbanPop        Rape
Alabama    1.24256408 0.7828393 -0.5209066 -0.003416473
Alaska     0.50786248 1.1068225 -1.2117642 2.484202941
Arizona    0.07163341 1.4788032 0.9989801 1.042878388
Arkansas   0.23234938 0.2308680 -1.0735927 -0.184916602
California 0.27826823 1.2628144 1.7589234 2.067820292
Colorado   0.02571456 0.3988593 0.8608085 1.864967207
```

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> #Cluster the dataset with 5 groups

> print(km)

K-means clustering with 5 clusters of sizes 7, 10, 10, 11, 12

Cluster means:

	Murder	Assault	UrbanPop	Rape
1	1.5803956	0.9662584	-0.7775109	0.04844071
2	-1.1727674	-1.2078573	-1.0045069	-1.10202608
3	-0.6286291	-0.4086988	0.9506200	-0.38883734
4	-0.1642225	-0.3658283	-0.2822467	-0.11697538
5	0.7298036	1.1188219	0.7571799	1.32135653

Clustering vector:

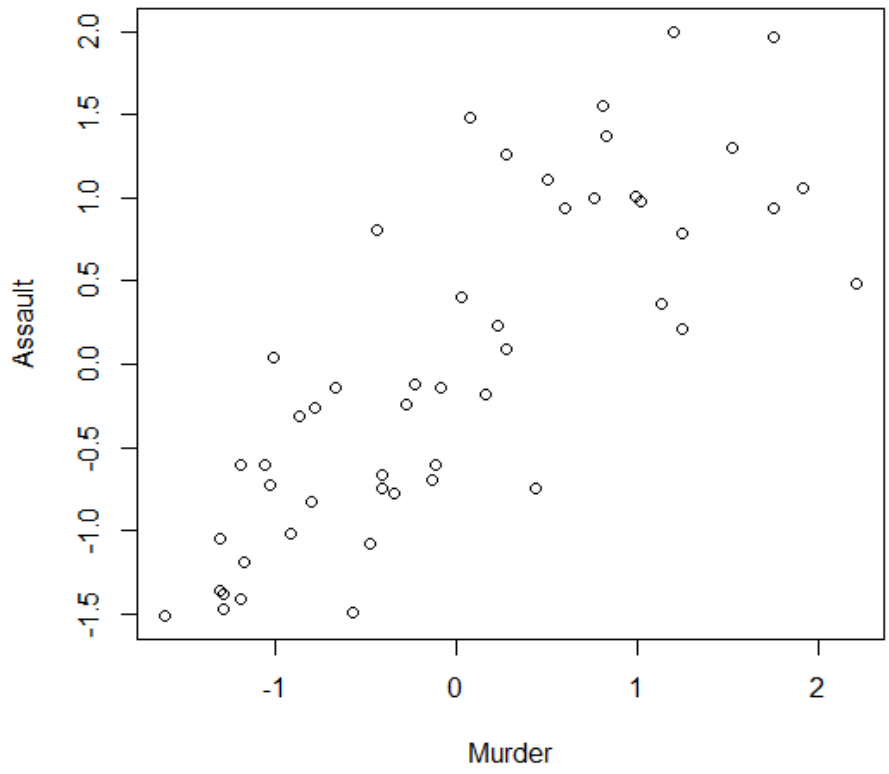
Alabama	Alaska	Arizona	Arkansas	California
1	5	5	4	5
Colorado	**Connecticut**	**Delaware**	**Florida**	**Georgia**
5	3	3	5	1
Hawaii	**Idaho**	**Illinois**	**Indiana**	**Iowa**
3	2	5	4	2
Kansas	**Kentucky**	**Louisiana**	**Maine**	**Maryland**
4	4	1	2	5
Massachusetts	**Michigan**	**Minnesota**	**Mississippi**	**Missouri**
3	5	2	1	4
Montana	**Nebraska**	**Nevada**	**New Hampshire**	**New Jersey**
4	4	5	2	3
New Mexico	**New York**	**North Carolina**	**North Dakota**	**Ohio**
5	5	1	2	3
Oklahoma	**Oregon**	**Pennsylvania**	**Rhode Island**	**South Carolina**
4	4	3	3	1

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

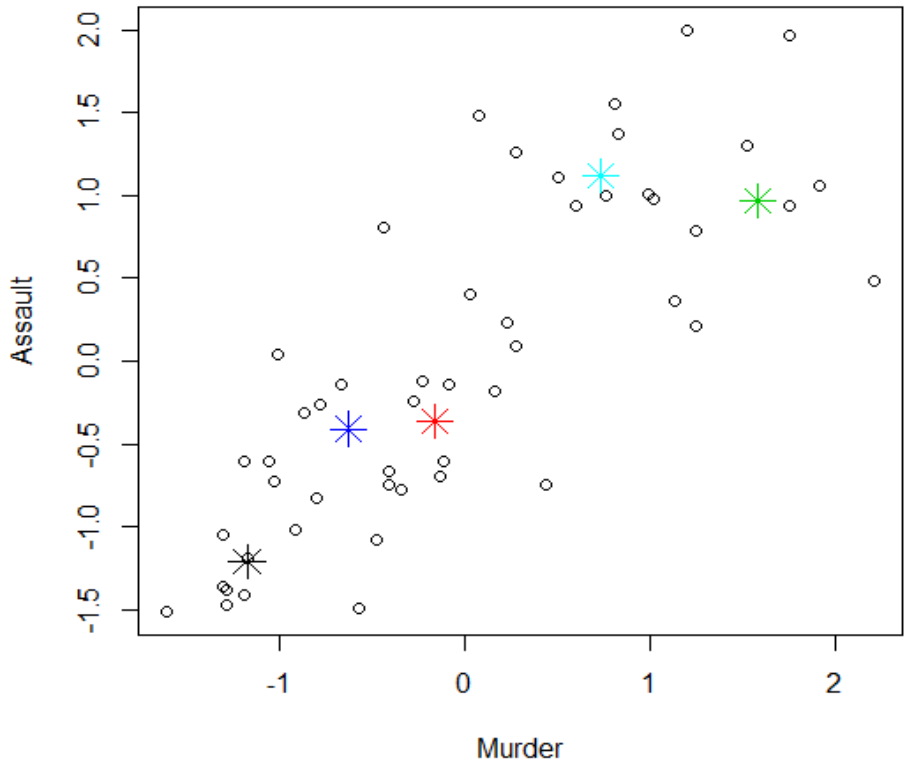
```
South Dakota     Tennessee        Texas        Utah       Vermont
       2            1             5            3            2
Virginia     Washington   West Virginia     Wisconsin       Wyoming
       4            3             2            2            4
Within cluster sum of squares by cluster:
[1]  6.128432  7.443899  9.326266  7.788275 18.257332
(between_SS / total_SS = 75.0 %)
Available components:
[1] "cluster"      "centers"      "totss"        "withinss"     "tot.withinss"
[6] "betweenss"    "size"         "iter"         "ifault"
```

> plot(df)



Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> points(km$centers,col=1:5,pch=8,cex=2)



> print(cnt)

1 2 3 4 5

7 10 10 11 12

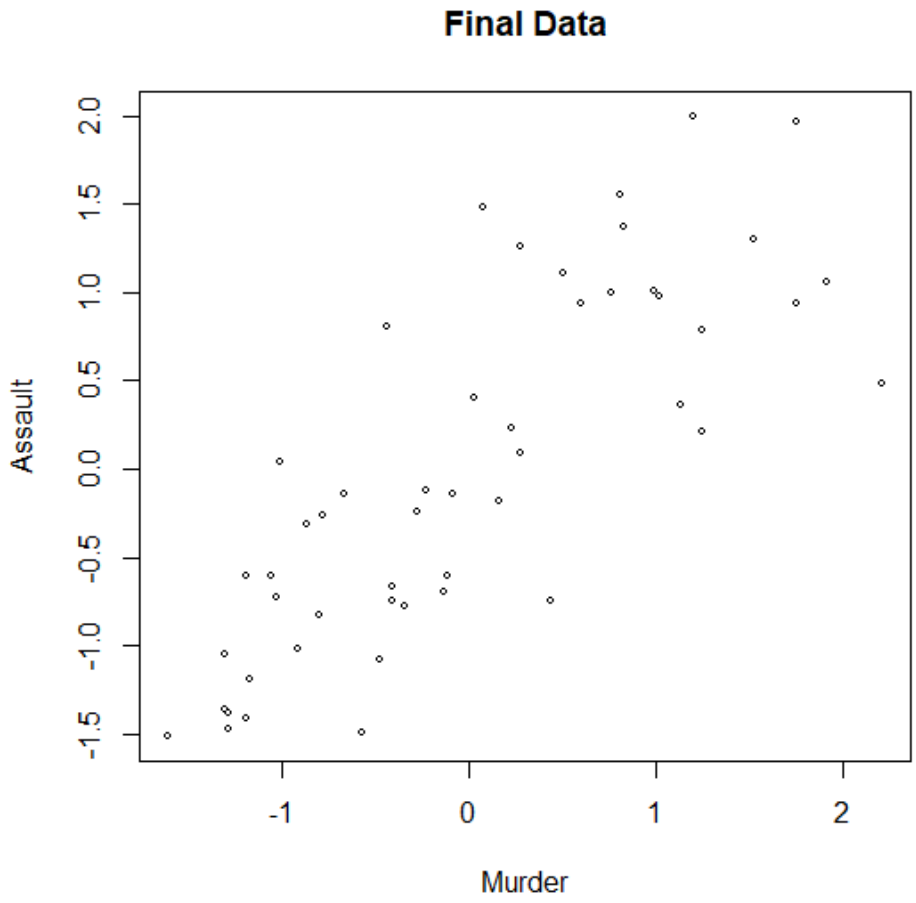
> head(final_data)

	Murder	Assault	UrbanPop	Rape	cluster
Alabama	1.24256408	0.7828393	-0.5209066	-0.003416473	1
Alaska	0.50786248	1.1068225	-1.2117642	2.484202941	5
Arizona	0.07163341	1.4788032	0.9989801	1.042878388	5
Arkansas	0.23234938	0.2308680	-1.0735927	-0.184916602	4
California	0.27826823	1.2628144	1.7589234	2.067820292	5
Colorado	0.02571456	0.3988593	0.8608085	1.864967207	5

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> plot(final_data,cex=0.6,main="Final Data")

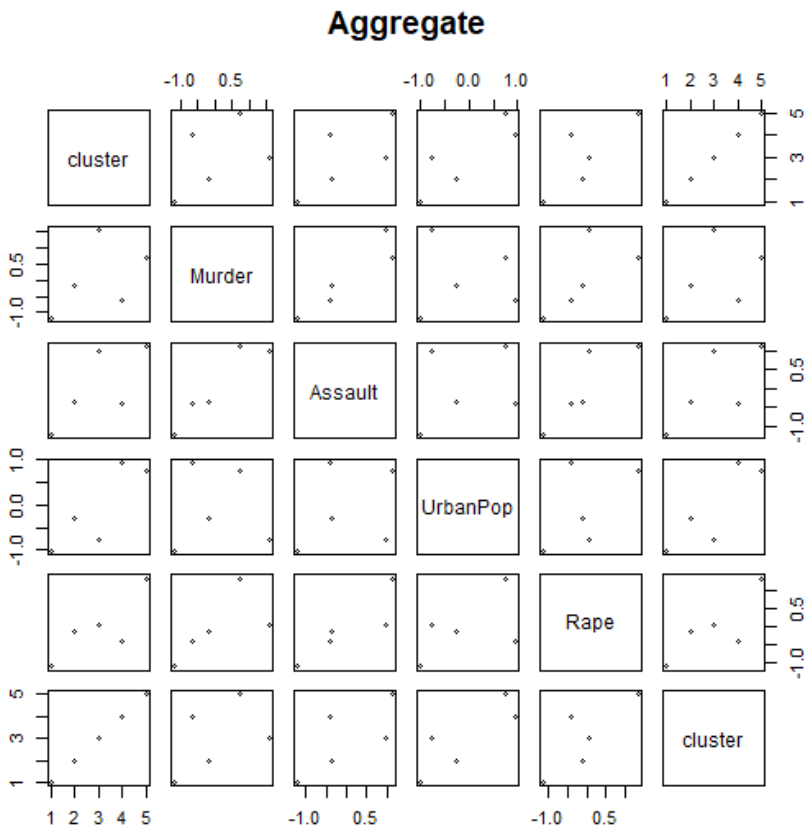


> head(ag)

	cluster	Murder	Assault	UrbanPop	Rape	cluster
1	1	1.5803956	0.9662584	-0.7775109	0.04844071	1
2	2	-1.1727674	-1.2078573	-1.0045069	-1.10202608	2
3	3	-0.6286291	-0.4086988	0.9506200	-0.38883734	3
4	4	-0.1642225	-0.3658283	-0.2822467	-0.11697538	4
5	5	0.7298036	1.1188219	0.7571799	1.32135653	5

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> plot(ag,cex=0.6,main="Aggregate")



RESULT:

Thus the program was executed successfully and the output is verified.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

EX. NO: 03	HIERARCHICAL CLUSTERING	PAGE NO:
DATE :		

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

3. HIERARCHICAL CLUSTERING

```
#Hierarchical Clustering
library(cluster)
df=USArrests
#remove rows with missing values
df=na.omit(df)
#scale each variable to have a mean 0 and sd of 1
df=scale(df)
head(df)
d=dist(df,method="euclidean")
#complete dendogram
hc1=hclust(d,method="complete")
plot(hc1,cex=0.6,main="complete dendogram",hang=-1)
#average dendogram
hc2=hclust(d,method="average")
plot(hc2,cex=0.6,main="Average Dendogram",hang=-1)
abline(h=3.0,col="green")
groups=cutree(hc2,k=4)
print(groups)
table(groups)
rect.hclust(hc2,k=4,border="red")
final_data=cbind(df,cluster=groups)
head(final_data)
plot(final_data,cex=0.6,main="Final Data")
```

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

OUTPUT:

#Hierarchical Clustering

> #remove rows with missing values

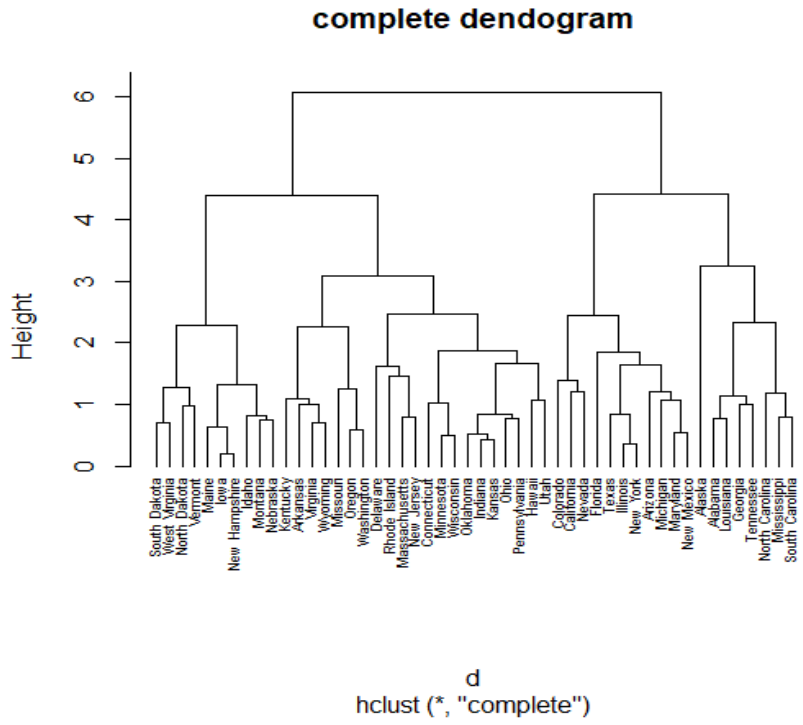
> #scale each variable to have a mean 0 and sd of 1

> head(df)

	Murder	Assault	UrbanPop	Rape
Alabama	1.24256408	0.7828393	-0.5209066	-0.003416473
Alaska	0.50786248	1.1068225	-1.2117642	2.484202941
Arizona	0.07163341	1.4788032	0.9989801	1.042878388
Arkansas	0.23234938	0.2308680	-1.0735927	-0.184916602
California	0.27826823	1.2628144	1.7589234	2.067820292
Colorado	0.02571456	0.3988593	0.8608085	1.864967207

> #complete dendogram

> plot(hc1,cex=0.6,main="complete dendogram",hang=-1)



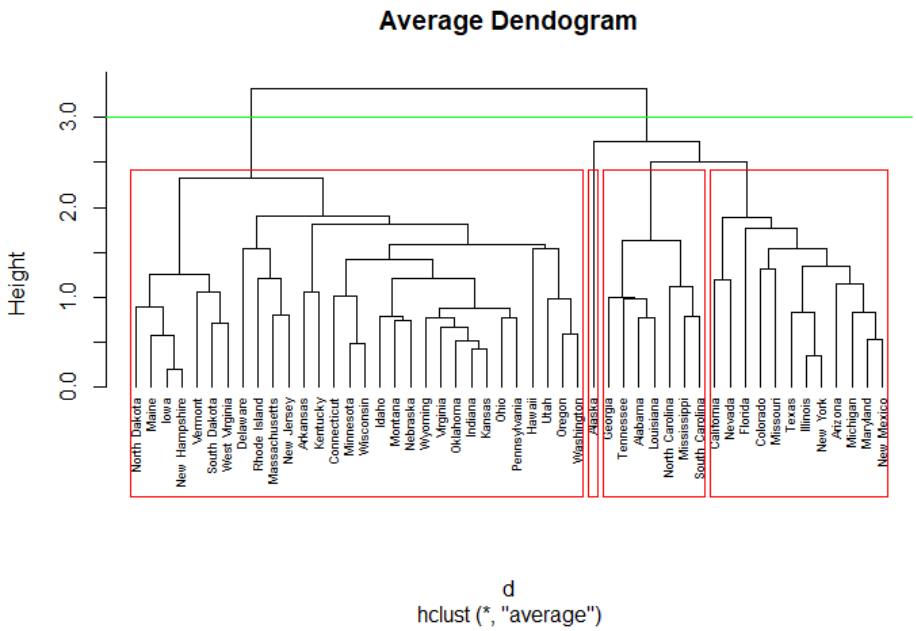
Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

```
> #average dendogram
> print(groups)
     Alabama       Alaska      Arizona     Arkansas    California
           1            2            3            4            3
    Colorado   Connecticut     Delaware      Florida      Georgia
           3            4            4            3            1
      Hawaii        Idaho     Illinois      Indiana         Iowa
           4            4            3            4            4
      Kansas     Kentucky    Louisiana        Maine     Maryland
           4            4            1            4            3
    Massachusetts     Michigan     Minnesota   Mississippi     Missouri
           4            3            4            1            3
      Montana     Nebraska       Nevada  New  Hampshire    New Jersey
           4            4            3            4            4
      New Mexico     New York North Carolina   North Dakota         Ohio
           3            3            1            4            4
    Oklahoma       Oregon   Pennsylvania   Rhode Island South Carolina
           4            4            4            4            1
    South Dakota    Tennessee        Texas         Utah      Vermont
           4            1            3            4            4
    Virginia    Washington   West  Virginia     Wisconsin      Wyoming
           4            4            4            4            4
groups
 1  2  3  4
 7  1 12 30
```

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> rect.hclust(hc2,k=4,border="red")



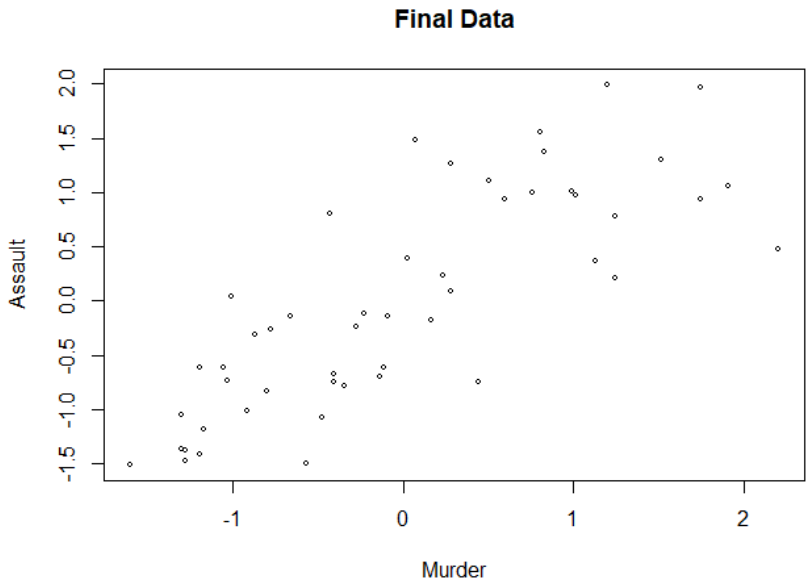
> final_data=cbind(df,cluster=groups)

> head(final_data)

	Murder	Assault	UrbanPop	Rape	cluster
Alabama	1.24256408	0.7828393	-0.5209066	-0.003416473	1
Alaska	0.50786248	1.1068225	-1.2117642	2.484202941	2
Arizona	0.07163341	1.4788032	0.9989801	1.042878388	3
Arkansas	0.23234938	0.2308680	-1.0735927	-0.184916602	4
California	0.27826823	1.2628144	1.7589234	2.067820292	3
Colorado	0.02571456	0.3988593	0.8608085	1.864967207	3

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> plot(final_data,cex=0.6,main="Final Data")



RESULT:

Thus the program was executed successfully and the output is verified.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

EX. NO: 04	CLASSIFICATION ALGORITHM	PAGE NO:
DATE :		

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

4. CLASSIFICATION ALGORITHM

```
#Classification Algorithm
library(class)
df=data(iris)
#Number of Rows and Columns
dim(iris)
head(iris)
rand=sample(1:nrow(iris),0.9*nrow(iris))
head(rand)
#Scale the values using Normalization method
nor<-function(x)
{
return((x-min(x))/(max(x)-min(x)))
}
iris_norm=as.data.frame(lapply(iris[,c(1,2,3,4)],nor))
head(iris_norm)
#Train dataset
iris_train=iris_norm[rand,]
iris_train_target=iris[rand,5]
#Test dataset
iris_test=iris_norm[-rand,]
iris_test_target=iris[-rand,5]
dim(iris_train)
dim(iris_test)
#K-nearesr neighbour Classification
model1=knn(train=iris_train,test=iris_test,cl=iris_train_target,k=7)
```

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

#Confusion Matric

```
tab=table(model1,iris_test_target)
print(tab)
accuracy=function(x)
{
sum(diag(x)/sum(rowSums(x)))*100
}
cat("Accuracy classifier=",accuracy(tab))
```

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

OUTPUT:

```
#Classification Algorithm
> #Number of Rows and Columns
> dim(iris)
[1] 150   5
> head(iris)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
> head(rand)
[1] 114 107  25 128  14  24
> #Scale the values using Normalization method
> nor<-function(x)
+ {
+ return((x-min(x))/(max(x)-min(x)))
+ }
> head(iris_norm)
  Sepal.Length Sepal.Width Petal.Length Petal.Width
1   0.22222222   0.6250000   0.06779661  0.04166667
2   0.16666667   0.4166667   0.06779661  0.04166667
3   0.11111111   0.5000000   0.05084746  0.04166667
4   0.08333333   0.4583333   0.08474576  0.04166667
```

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

```
5   0.19444444   0.6666667   0.06779661  0.04166667
6   0.30555556   0.7916667   0.11864407  0.12500000
> #Train dataset
> iris_train=iris_norm[rand,]
> iris_train_target=iris[rand,5]
> #Test dataset
> iris_test=iris_norm[-rand,]
> iris_test_target=iris[-rand,5]
> dim(iris_train)
[1] 135   4
> dim(iris_test)
[1] 15  4
> #K-nearesr neighbour Classification
> model1=knn(train=iris_train,test=iris_test,cl=iris_train_target,k=7)
> #Confusion Matric
> print(tab)
           iris_test_target
model1       setosa versicolor virginica
  setosa          6          0         0
  versicolor      0          6         1
  virginica       0          0         2
> accuracy=function(x)
+ {
+ sum(diag(x)/sum(rowSums(x)))*100
+ }
> cat("Accuracy classifier=",accuracy(tab))
Accuracy classifier= 100>
```

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

RESULT:

Thus the program was executed successfully and the output is verified.

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

EX. NO: 05	DECISION TREE	PAGE NO:
DATE :		

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

5. DECISION TREE

```
#Decision Tree
library(rpart)
data=iris
str=data
head(data)
#creating the decision tree using regression
dtree=rpart(Sepal.Width~Sepal.Length+Petal.Width+Petal.Length+Species,data=iris,method="anova")
plot(dtree,uniform=TRUE,main="Sepal Width Decision Tree Using Regression")
print(dtree)
text(dtree,use.n=TRUE,cex=.7)
#predicting the Sepal Width
adata<-
data.frame(Species='versicolor',Sepal.Length=5.1,Petal.Length=4.5,Petal.Width=1.4)
cat("Predicted Value:\n")
pt=predict(dtree,adata,method="anova")
print(pt)
plot(pt)
#creating the decision tree using classification
df=as.data.frame(data)
dt=rpart(Sepal.Width~Sepal.Length+Petal.Width+Petal.Length+Species,data=df,method="class")
plot(dt,uniform=TRUE,main="Sepal Width Decision Tree using Classification")
print(dt)
text(dt,use.n=TRUE,cex=.7)
```

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

OUTPUT:

> #Decision Tree

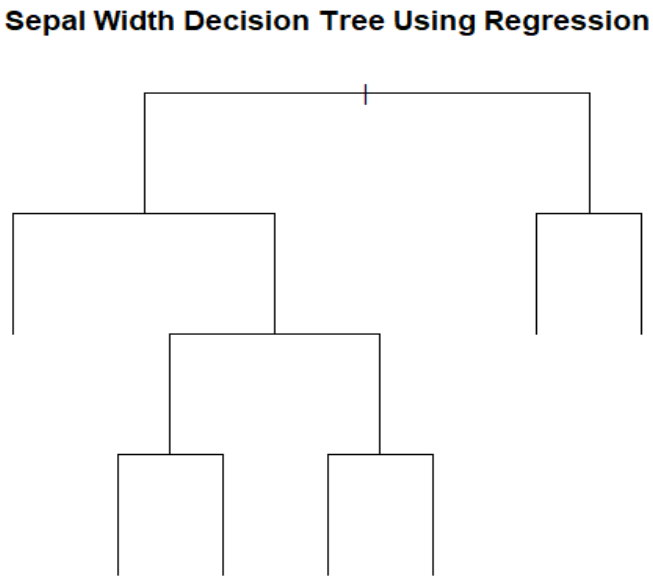
> head(data)

	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
1	5.1	3.5	1.4	0.2	setosa
2	4.9	3.0	1.4	0.2	setosa
3	4.7	3.2	1.3	0.2	setosa
4	4.6	3.1	1.5	0.2	setosa
5	5.0	3.6	1.4	0.2	setosa
6	5.4	3.9	1.7	0.4	setosa

> #creating the decision tree using regression

dtree=rpart(Sepal.Width~Sepal.Length+Petal.Width+Petal.Length+Species,data=iris,method="anova")

> plot(dtree,uniform=TRUE,main="Sepal Width Decision Tree Using Regression")



Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> print(dtree)

n= 150

node), split, n, deviance, yval

* denotes terminal node

1) root 150 28.3069300 3.057333

2) Species=versicolor,virginica 100 10.9616000 2.872000

4) Petal.Length< 4.05 16 0.7975000 2.487500 *

5) Petal.Length>=4.05 84 7.3480950 2.945238

10) Petal.Width< 1.95 55 3.4920000 2.860000

20) Sepal.Length< 6.35 36 2.5588890 2.805556 *

21) Sepal.Length>=6.35 19 0.6242105 2.963158 *

11) Petal.Width>=1.95 29 2.6986210 3.106897

22) Petal.Length< 5.25 7 0.3285714 2.914286 *

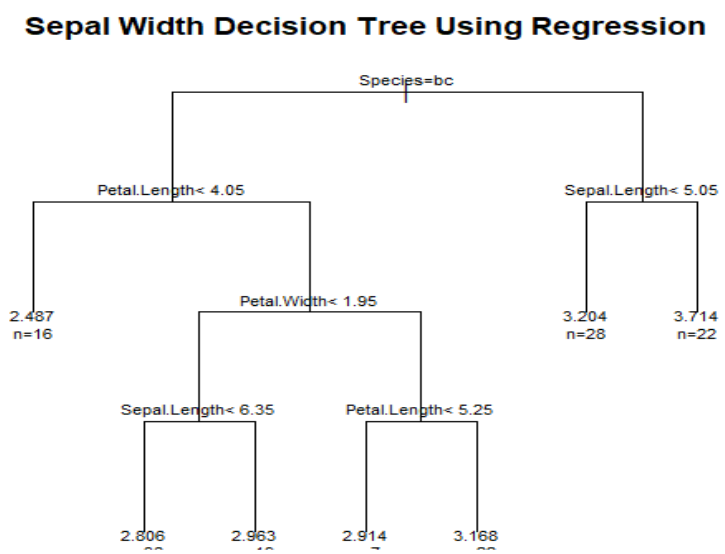
23) Petal.Length>=5.25 22 2.0277270 3.168182 *

3) Species=setosa 50 7.0408000 3.428000

6) Sepal.Length< 5.05 28 2.0496430 3.203571 *

7) Sepal.Length>=5.05 22 1.7859090 3.713636 *

> text(dtree,use.n=TRUE,cex=.7)



This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

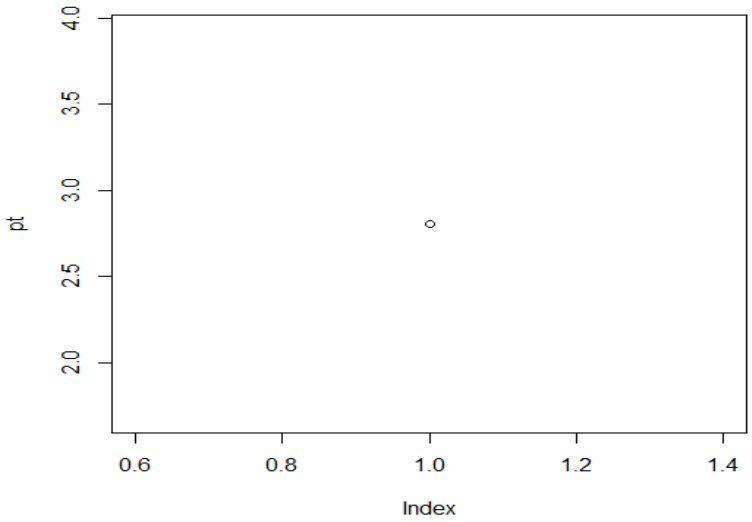
> #predicting the Sepal Width

Predicted Value:

1

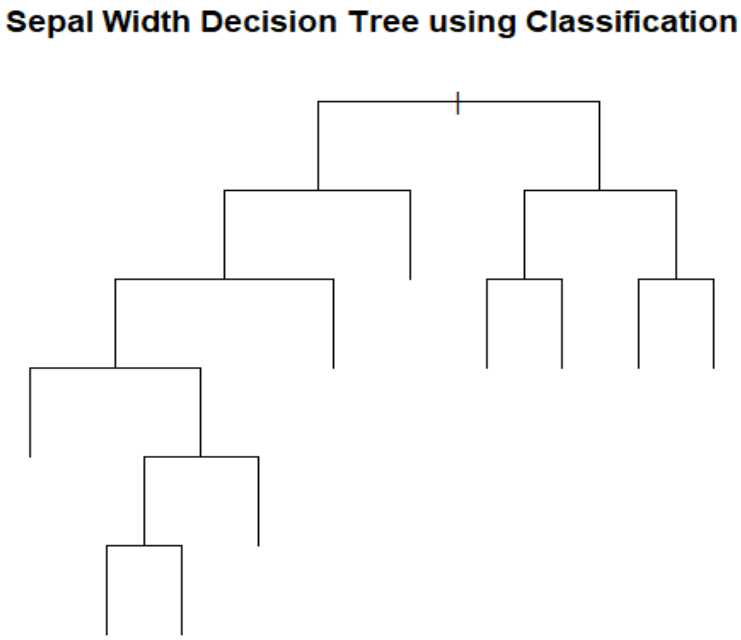
2.805556

> plot(pt)



> #creating the decision tree using classification

> plot(dt,uniform=TRUE,main="Sepal Width Decision Tree using Classification")



Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> print(dt)

n= 150

node), split, n, loss, yval, (yprob)

* denotes terminal node

1) root 150 124 3 (0.0067 0.02 0.027 0.02 0.053 0.033 0.06 0.093 0.067 0.17 0.073 0.087 0.04 0.08 0.04 0.027 0.02 0.04 0.013 0.0067 0.0067 0.0067 0.0067)

2) Petal.Width>=0.8 100 80 3 (0.01 0.03 0.03 0.03 0.08 0.05 0.09 0.14 0.09 0.2 0.07 0.08 0.04 0.03 0 0.01 0 0.02 0 0 0 0 0)

4) Sepal.Length< 6.45 65 55 2.8 (0.015 0.046 0.046 0.046 0.11 0.062 0.14 0.15 0.11 0.14 0.015 0.046 0.031 0.046 0 0 0 0 0 0 0 0 0)

8) Petal.Width< 1.95 56 47 2.7 (0.018 0.054 0.054 0.054 0.11 0.071 0.16 0.11 0.12 0.16 0.018 0.036 0.018 0.018 0 0 0 0 0 0 0 0 0)

16) Sepal.Length< 5.55 12 9 2.4 (0.083 0 0.17 0.25 0.25 0.083 0.083 0 0 0.083 0 0 0 0 0 0 0 0 0 0 0 0 0) *

17) Sepal.Length>=5.55 44 36 2.7 (0 0.068 0.023 0 0.068 0.068 0.18 0.14 0.16 0.18 0.023 0.045 0.023 0.023 0 0 0 0 0 0 0 0 0)

34) Petal.Width< 1.55 29 23 2.9 (0 0.1 0.034 0 0.069 0.1 0.1 0.17 0.21 0.17 0 0.034 0 0 0 0 0 0 0 0 0 0 0)

68) Sepal.Length>=5.95 15 11 2.9 (0 0.2 0.067 0 0.067 0.067 0 0.2 0.27 0.067 0 0.067 0 0 0 0 0 0 0 0 0 0 0) *

69) Sepal.Length< 5.95 14 10 3 (0 0 0 0 0.071 0.14 0.21 0.14 0.14 0.29 0 0 0 0 0 0 0 0 0 0 0 0 0) *

35) Petal.Width>=1.55 15 10 2.7 (0 0 0 0 0.067 0 0.33 0.067 0.067 0.2 0.067 0.067 0.067 0.067 0 0 0 0 0 0 0 0 0) *

9) Petal.Width>=1.95 9 5 2.8 (0 0 0 0 0.11 0 0 0.44 0 0 0 0.11 0.11 0.22 0 0 0 0 0 0 0 0 0) *

5) Sepal.Length>=6.45 35 24 3 (0 0 0 0 0.029 0.029 0 0.11 0.057 0.31 0.17 0.14 0.057 0 0 0.029 0 0.057 0 0 0 0 0) *

3) Petal.Width< 0.8 50 41 3.4 (0 0 0.02 0 0 0 0 0 0.02 0.12 0.08 0.1 0.04 0.18 0.12 0.06 0.06 0.08 0.04 0.02 0.02 0.02 0.02)

6) Sepal.Length< 4.95 20 15 3 (0 0 0.05 0 0 0 0 0 0.05 0.25 0.2 0.2 0 0.15 0 0.1 0 0 0 0 0 0 0)

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

12) Petal.Length< 1.45 13 8 3 (0 0 0.077 0 0 0 0 0 0.077 0.38 0 0.23 0 0.077 0 0.15 0 0 0 0 0 0 0) *

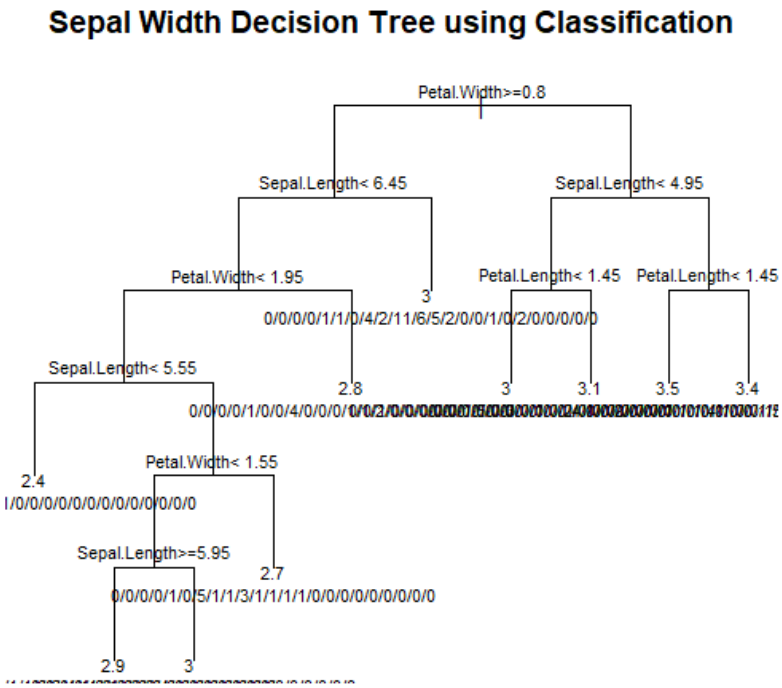
13) Petal.Length>=1.45 7 3 3.1 (0 0 0 0 0 0 0 0 0 0 0.57 0.14 0 0.29 0 0 0 0 0 0 0 0 0) *

7) Sepal.Length>=4.95 30 24 3.4 (0 0 0 0 0 0 0 0 0 0.033 0 0.033 0.067 0.2 0.2 0.033 0.1 0.13 0.067 0.033 0.033 0.033 0.033)

14) Petal.Length< 1.45 11 7 3.5 (0 0 0 0 0 0 0 0 0 0 0 0.091 0.091 0.091 0.36 0.091 0 0 0.091 0.091 0 0.091 0) *

15) Petal.Length>=1.45 19 14 3.4 (0 0 0 0 0 0 0 0 0 0.053 0 0 0.053 0.26 0.11 0 0.16 0.21 0.053 0 0.053 0 0.053) *

> text(dt,use.n=TRUE,cex=.7)



RESULT:

Thus the program was executed successfully and the output is verified.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

EX. NO: 06	LINEAR REGRESSION	PAGE NO:
DATE :		

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

6. LINEAR REGRESSION

```
#Linear Regression
setwd("D:/R")
df=read.csv("h2.csv",header=TRUE)
print(df)
lr=lm(height~weight,data=df)
print(lr)
#Linear Regression
plot(df$height,df$weight,col="blue",main="Height_Weight
Regression",cex=1.3,pch=15,xlab="height",ylab="weight")
print(summary(lr))
print(residuals(lr))
coeff=coefficients(lr)
eq=paste0("y",round(coeff[2],1),"*(",round(coeff[1],1),"*x)")
print(eq)
#Linear Equation
new.weights=data.frame(weight=c(60,50))
print(new.weights)
df1=predict(lr,newdata=new.weights)
print(df1)
df2=data.frame(df1,new.weights)
names(df2)=c("height","weight")
print(df2)
df3=rbind(df,df2)
print(df3)
write.csv(df3,"h3.csv")
pie(table(df3$height))
```

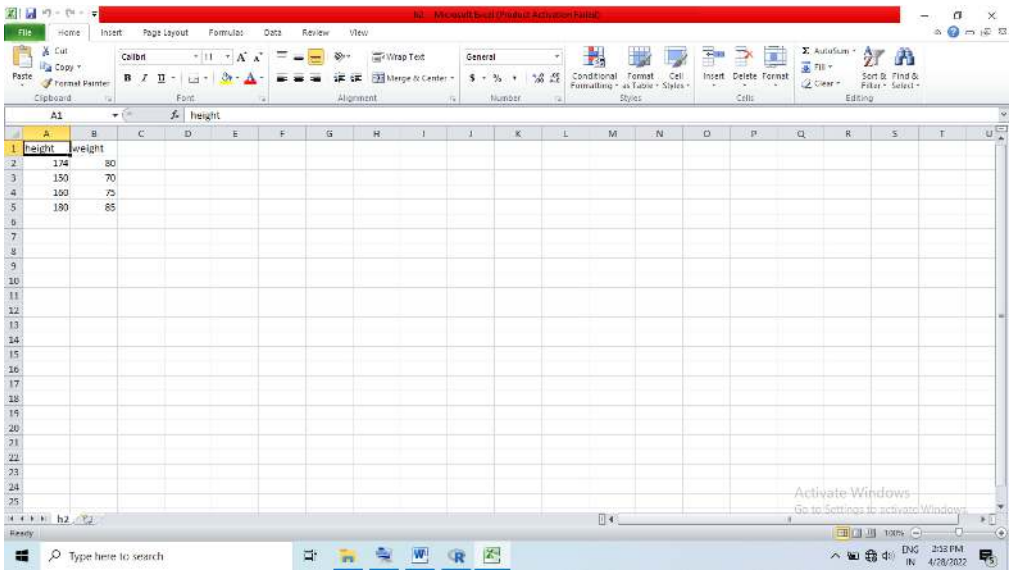
Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

OUTPUT:

> #Linear Regression

> setwd("D:/R")

> df=read.csv("h2.csv",header=TRUE)



> print(df)

height weight

1 174 80

2 150 70

3 160 75

4 180 85

> lr=lm(height~weight,data=df)

> print(lr)

Call:

lm(formula = height ~ weight, data = df)

Coefficients:

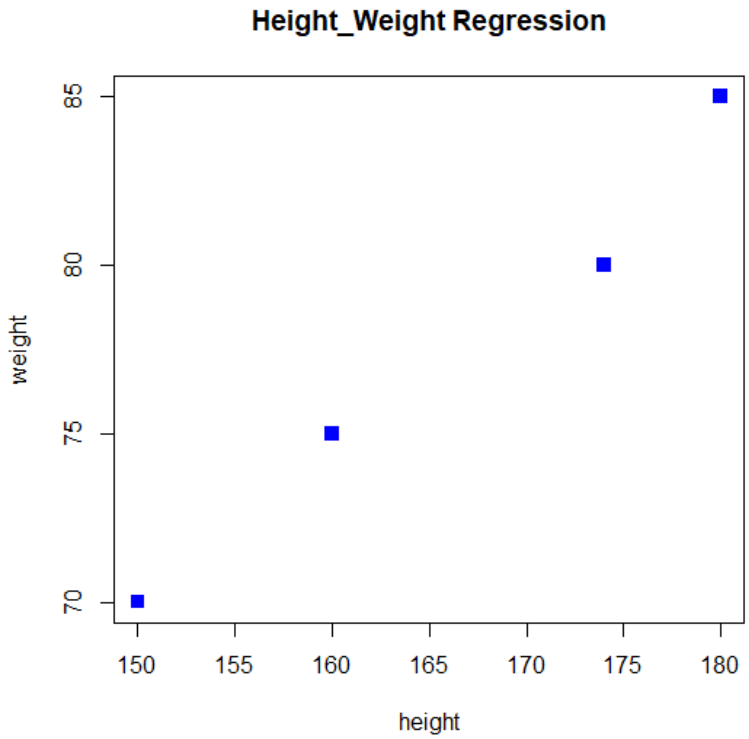
(Intercept) weight

4.80 2.08

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> #Linear Regression



> print(summary(lr))

Call:

lm(formula = height ~ weight, data = df)

Residuals:

```
   1    2    3    4
 2.8 -0.4 -0.8 -1.6
```

Coefficients:

```
            Estimate Std. Error t value Pr(>|t|)
(Intercept)   4.8000    16.4463   0.292   0.7979
weight        2.0800     0.2117   9.827   0.0102 *
---
Signif. codes:  0 ‘***’ 0.001 ‘**’ 0.01 ‘*’ 0.05 ‘.’ 0.1 ‘ ’ 1
```

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

```
Residual standard error: 2.366 on 2 degrees of freedom
Multiple R-squared:  0.9797,    Adjusted R-squared:  0.9696
F-statistic: 96.57 on 1 and 2 DF,  p-value: 0.0102
> print(residuals(lr))
   1    2    3    4
 2.8 -0.4 -0.8 -1.6
> print(eq)
[1] "y2.1*(4.8*x)"
> #Linear Equation
> print(new.weights)
  weight
1     60
2     50
> print(df1)
    1     2
129.6 108.8
> print(df2)
  height weight
1  129.6     60
2  108.8     50
> df3=rbind(df,df2)
> print(df3)
   height weight
1   174.0     80
2   150.0     70
3   160.0     75
4   180.0     85
```

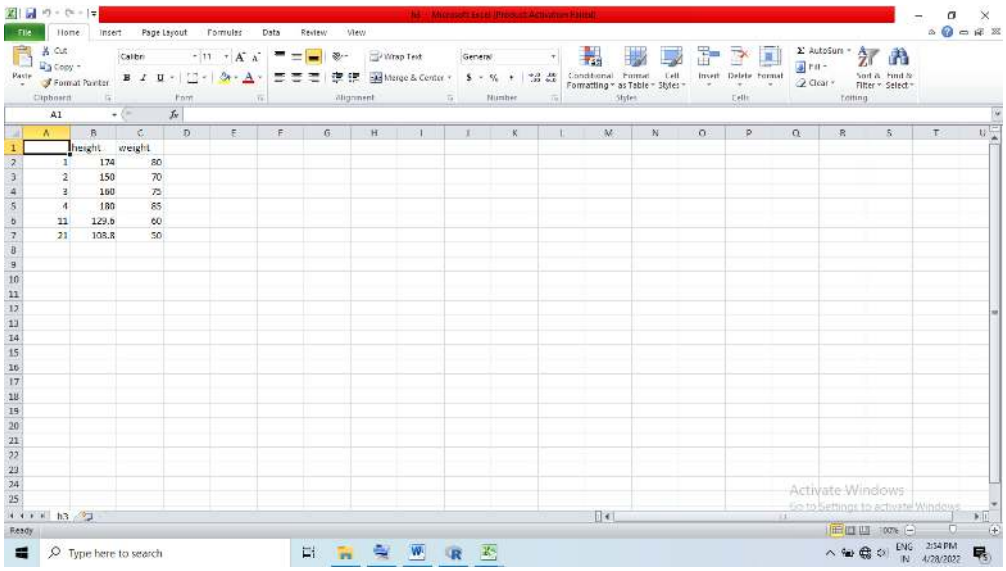
This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

11 129.6 60

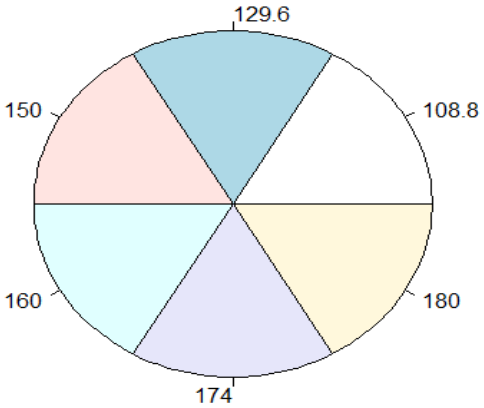
21 108.8 50

> write.csv(df3,"h3.csv")



	height	weight
1	174	80
2	150	70
3	160	75
4	180	85
11	129.6	60
21	108.8	50

> pie(table(df3$height))



RESULT:

Thus the program was executed successfully and the output is verified.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

EX. NO: 07	DATA VISUALIZATION	PAGE NO:
DATE :		

This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

7. DATA VISUALIZATION

```
#Data Visualization
X=iris
dim(X)
summary(X)
head(X)
hist(X$Sepal.Length,main='Histogram',col='green')
barplot(X$Sepal.Length[1:10],main='Barplot',col='red',xlab='Sepal.Length')
pie(table(X$Sepal.Length),main='pie-chart')
pairs(X)
plot(X$Sepal.Length,main='plot-chart',col='blue')
boxplot(X,main='Boxplot',col='yellow')
```

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

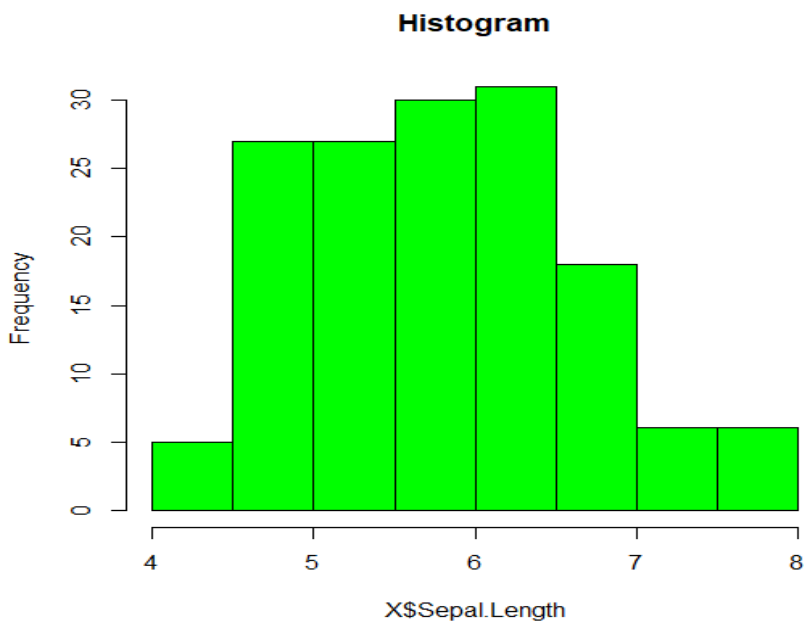
OUTPUT:

```
> #Data Visualization
> dim(X)
[1] 150   5
> summary(X)
  Sepal.Length    Sepal.Width     Petal.Length
 Min.   :4.300   Min.   :2.000   Min.   :1.000
 1st Qu.:5.100   1st Qu.:2.800   1st Qu.:1.600
 Median:5.800   Median:3.000   Median:4.350
 Mean   :5.843   Mean   :3.057   Mean   :3.758
 3rd Qu.:6.400   3rd Qu.:3.300   3rd Qu.:5.100
 Max.   :7.900   Max.   :4.400   Max.   :6.900
  Petal.Width          Species
 Min.   :0.100   setosa     :50
 1st Qu.:0.300   versicolor:50
 Median:1.300   virginica :50
 Mean   :1.199
 3rd Qu.:1.800
 Max.   :2.500
> head(X)
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa
```

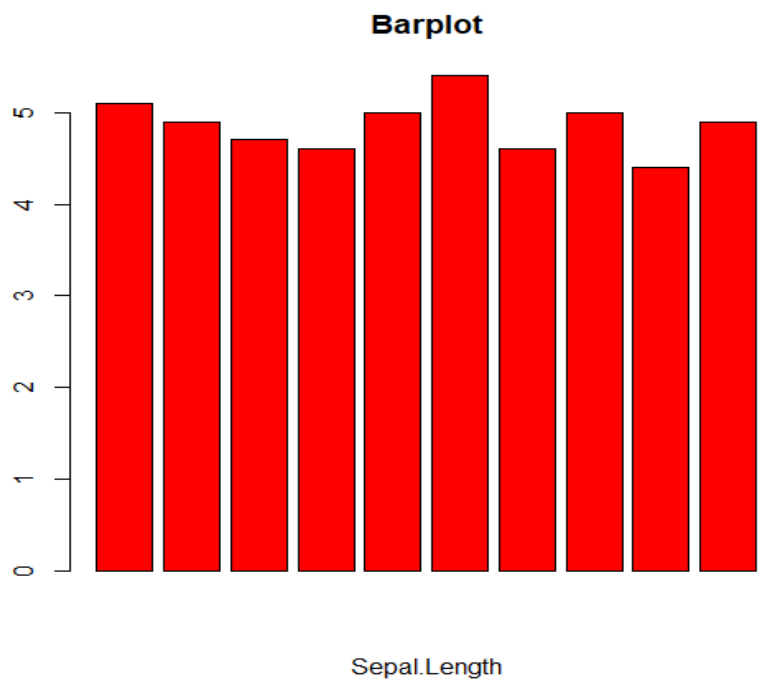
This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> hist(X$Sepal.Length,main='Histogram',col='green')

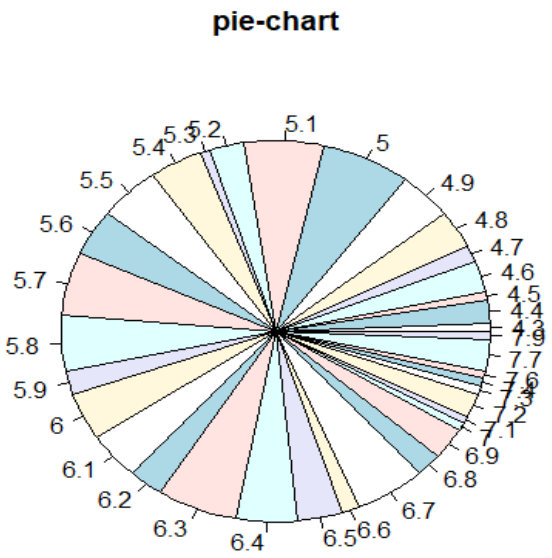


> barplot(X$Sepal.Length[1:10],main='Barplot',col='red',xlab='Sepal.Length')

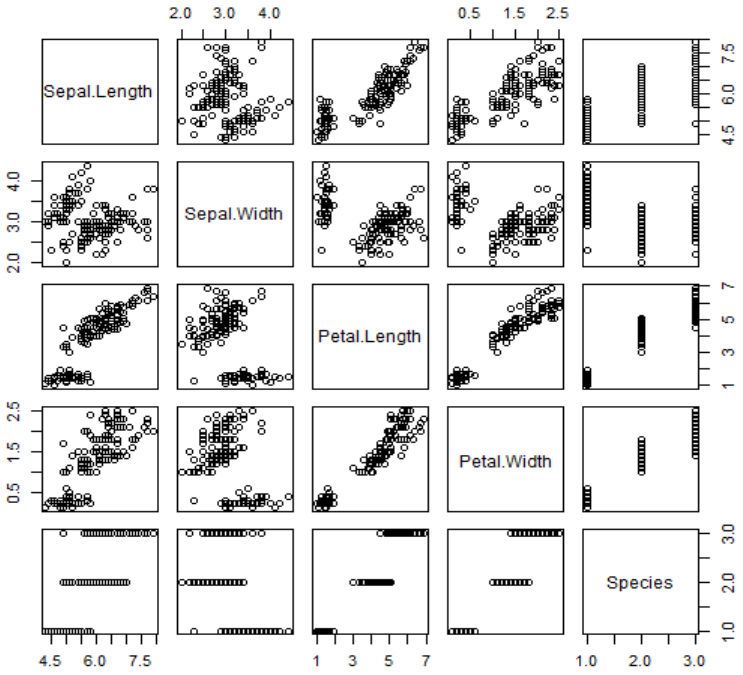


Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> pie(table(X$Sepal.Length),main='pie-chart')



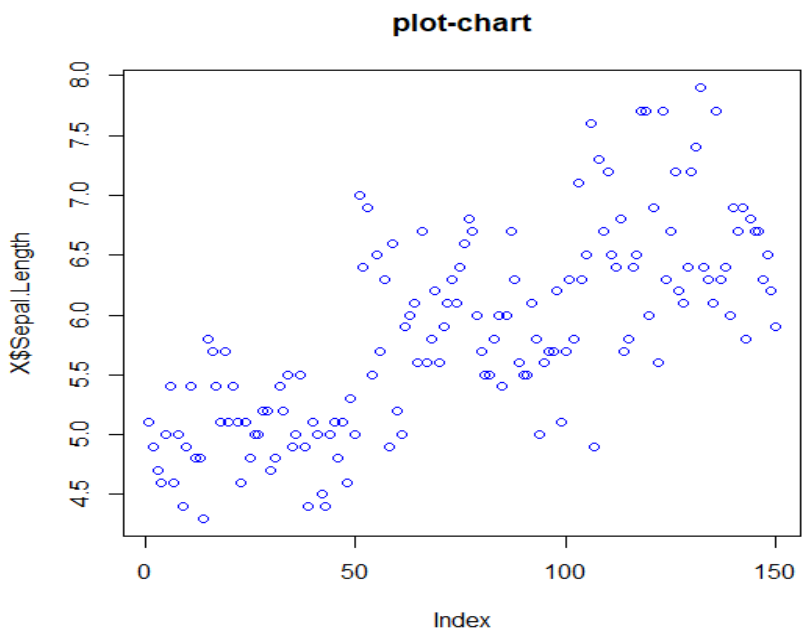
> pairs(X)



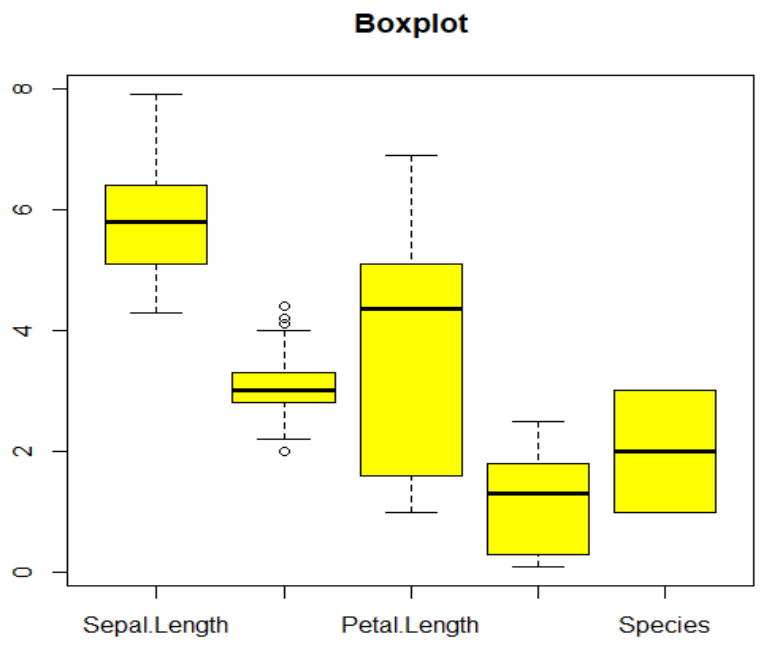
This document is available on studocu

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)

> plot(X$Sepal.Length,main='plot-chart',col='blue')



> boxplot(X,main='Boxplot',col='yellow')



<u>RESULT:</u>

Thus the program was executed successfully and the output is verified.

Downloaded by bhagavathi 0411 (bhagavathi04112003@gmail.com)